TPU v6e Precision Formats

https://gemini.google.com/app/e94ef97bc1bfdee3

User prompt: What is the best precision format to run on the TPU v6e? I'm only talking about one TPU. That means with only 16 GB of VRAM. I will be using this through Google Colab. It is currently available on the Colab program.

Response: A single **TPU v6e (Trillium)** chip physically features **32 GB of High-Bandwidth Memory (HBM)**, while its predecessor, the TPU v5e, is the model equipped with 16 GB of HBM. Whether you are operating within a 16 GB boundary or taking advantage of the full 32 GB on v6e in Google Colab, the optimal precision formats are **`bfloat16` (BF16)** for general tasks and **`FP8` (E4M3)** for memory-constrained inference.

Precision Format Comparison on TPU v6e

Format	Native MXU Support	Peak Compute (Per Chip)	Memory / Param	Best Use Case
`bfloat16` (BF16)	Yes (Native)	918 TFLOPs	2 Bytes	**Default recommendation**: Training, LoRA fine-tuning, general inference.
`FP8` (`E4M3` / `E5M2`)	Yes (Native)	~1,836 TFLOPs	1 Byte	**Memory-constrained serving**: Maximizing batch size, context length, and throughput.
`int8`	Yes (Native)	1,836 TOPs	1 Byte	Integer quantization (W8A8 / W8A16) where FP8 is unavailable.
`float16` (FP16)	Emulated / Downcast	Degraded	2 Bytes	**Avoid on TPU**: Prone to numerical underflow/overflow; lacks native systolic paths.
`float32` (FP32)	Accumulated only	Degraded	4 Bytes	Master weights, loss computation, optimizer states only.

1. The Default Choice: `bfloat16` (BF16)

`bfloat16` is the native format for Google TPUs. The TPU v6e's Matrix Multiply Units (MXUs) execute systolic multiplications directly in BF16 and accumulate results in FP32 at the hardware level.

- **Dynamic Range:** Retains the exact same 8-bit dynamic range as `float32`, eliminating the need for loss scaling factors or overflow safeguards.
- **Workload Fit:** Ideal for training, QLoRA/LoRA tuning, embeddings, and general LLM generation.
- **Model Sizing:** An 8B parameter model in BF16 consumes ~16 GB just for model weights, leaving tight headroom on a 16 GB allocation but running comfortably within v6e's native 32 GB HBM.

2. High-Throughput & Low-VRAM Inference: `FP8`

The TPU v6e architecture introduced native hardware datapaths for **8-bit floating point (FP8)** in the MXU.

- **Compute & Bandwidth:** Doubles the raw compute throughput compared to BF16 (up to ~1,836 TFLOPs) and cuts HBM memory read pressure by 50%.
- **Which FP8 Variant:**
  - **`E4M3` (1 sign, 4 exponent, 3 mantissa bits):** Best for forward passes, inference activations, and weight matrices where preserving precision is prioritized over dynamic range.
  - **`E5M2` (1 sign, 5 exponent, 2 mantissa bits):** Mimics FP16 range; primarily used for gradient backpropagation if attempting FP8 training.
- **Implementation:** Supported natively in JAX via Google's `AQT` (Accurate Quantized Training) and in reference stacks like MaxText and vLLM-TPU.

3. Formats to Explicitly Avoid

- **`float16` (FP16):** Unlike NVIDIA GPUs where FP16 Tensor Cores were standard for years, TPUs do not have dedicated FP16 arithmetic pipelines. Standard FP16 often encounters numerical instability or triggers compiler overhead via XLA conversions.
- **`float32` (FP32):** For matrix operations, XLA automatically truncates FP32 math to BF16 or TF32 equivalents unless strict precision flags are forced. Storing active tensors in FP32 quadruples memory consumption relative to FP8 and will quickly exhaust single-chip Colab memory.

Configuring Precision in Colab

In **JAX**:

```
import jax
import jax.numpy as jnp

# Set default matrix multiplication precision for XLA compilation
jax.config.update("jax_default_matmul_precision", "bfloat16")

# Instantiate tensors/models in native BF16
x = jnp.ones((2048, 2048), dtype=jnp.bfloat16)
```

In **PyTorch / TorchTPU**:

```
import torch

# Ensure model parameters and inputs use torch.bfloat16
model = model.to(device="tpu", dtype=torch.bfloat16)
```

Sources

1. TPU v5e - Google Cloud Documentation
2. Google TPU v6e (Trillium) & XLA Architecture (2026) - Tech Interview
3. Tensor Processing Unit - Wikipedia

4. TPUs vs. GPUs: What They Are, How They Differ, and Which

User prompt: Yes, but then again, what is the precision format I should use for maximum token generation? These are the standard precision formats you're talking about. What will maximize token generation?

Response: To maximize token generation throughput and decode speed on a TPU v6e, the optimal configuration is **FP8 (`E4M3`) weights and activations combined with an FP8 KV cache**.

Why FP8 Maximizes Generation Speed

Autoregressive token generation operates under two distinct hardware constraints depending on whether you are processing the prompt or generating output tokens:

1. The Decode Bottleneck (Memory-Bandwidth Bound)

Generating tokens one by one at small batch sizes (e.g., batch size 1–4) is almost entirely bound by High-Bandwidth Memory (HBM) transfer speed, not compute capability. For every single token generated, the TPU must stream all active model weights from HBM into the compute registers:

$$\text{Decode Latency per Token} \approx \frac{\text{Model Weights Size (Bytes)} + \text{KV Cache Size (Bytes)}}{\text{HBM Memory Bandwidth}}$$

- **In `bfloat16`:** An 8B parameter model requires transferring ~16 GB across the memory bus for every token step.
- **In `FP8`:** The exact same model requires transferring only ~8 GB per step. Cutting the required memory bandwidth in half immediately approaches a $2\times$ **speedup in tokens/second** during autoregressive decoding.

2. The Prefill & High-Batch Bottleneck (Compute Bound)

During prompt processing (prefill) or when serving higher batch sizes, the matrix multiplication units (MXUs) become the bottleneck.

- TPU v6e has dedicated hardware datapaths that execute native FP8 tensor math at **1,836 TFLOPs** per chip—exactly **double the 918 TFLOPs of native `bfloat16`**.

3. Quantized KV Cache

As output sequences grow longer, the KV cache competes directly with the weights for HBM bandwidth:

- Standard 16-bit KV caching quickly eats up memory and slows down decode steps as context lengths cross 4k–8k tokens.
- Quantizing the KV cache to **FP8 (`E4M3` or `E5M2`)** cuts cache read traffic in half, keeps the cache within fast local memory, and allows you to run double the concurrent sequence batch size before spilling or hitting latency walls.

Precision Breakdown for Token Generation

Precision Setup	Decode Speed (Low Batch)	Prefill Throughput	TPU Hardware Execution	Generation Bottleneck
W8A8 FP8 + FP8 KV Cache	**Fastest** (~ $1.8\times$–$2\times$ vs BF16)	**Maximum** (1,836 TFLOPs)	Native MXU execution	Balanced
W4A16 (INT4 Weights, BF16 Activations)	Very Fast (lower HBM load)	Moderate	Dequantized via VPU to BF16	Compute overhead from on-the-fly dequantization
BF16 Weights + BF16 KV Cache	Baseline ($1\times$)	Baseline (918 TFLOPs)	Native MXU execution	Memory bandwidth bound
FP16	Slow	Poor	Emulated / XLA conversions	Underflow risks and compiler overhead

Why Not INT4 / W4A16?

On GPUs, 4-bit weight quantization (AWQ, GPTQ) is popular for maximizing batch-1 token generation. On TPUs, however, the v6e MXU does not have a native 4-bit matrix multiplier.

When running 4-bit weights on a TPU, the Vector Processing Unit (VPU) must dynamically dequantize the 4-bit integers back up to `bfloat16` or `FP8` before feeding them into the systolic array. This introduces instruction overhead inside XLA kernels that often degrades the memory-bandwidth gains. **W8A8 FP8 executes directly inside the systolic arrays without dequantization steps**, making it significantly more efficient end-to-end on TPU silicon.

Implementation in Colab

If serving models through **vLLM on TPU** or Google's **MaxText**:

- **vLLM-TPU engine flags:**

```
vllm serve <model_name> \
  --quantization fp8 \
  --kv-cache-dtype fp8 \
  --dtype bfloat16
```

- **JAX / AQT (Accurate Quantized Training & Inference):** Configure the dot-general operator to target `E4M3` for weights and activations, and downcast the attention layer's key-value projection tensors into `E4M3` prior to writing into the state cache.
